

Exercícios de autorrevisão

9.1 Preencha os espaços em cada uma das sentenças a seguir:

- a)** Toda entrada e saída é trabalhada na forma de _____.
- b)** O stream de _____ normalmente está conectado ao teclado.
- c)** O stream de _____ normalmente está conectado à tela do computador.
- d)** A formatação de saída precisa é obtida a partir da função _____.
- e)** A string de controle de formato pode conter _____, _____, _____ e _____.
- f)** Tanto o especificador de conversão _____ como o _____ pode ser usado para enviar um inteiro decimal com sinal.
- g)** Os especificadores de conversão _____, _____ e _____ são usados para exibir inteiros sem sinal em formato octal, decimal e hexadecimal, respectivamente.
- h)** Os modificadores _____ e _____ são colocados antes dos especificadores de conversão de inteiros para indicar que valores inteiros `short` ou `long` serão exibidos.
- i)** O especificador de conversão _____ é usado para exibir um valor de ponto flutuante na notação exponencial.
- j)** O modificador _____ é colocado antes de qualquer especificador de conversão de ponto flutuante para indicar que um valor `long double` deverá ser exibido.
- k)** Os especificadores de conversão `e`, `E` e `f` são exibidos com _____ dígitos de precisão à direita do ponto decimal, se nenhuma precisão for especificada.
- l)** Os especificadores de conversão _____ e _____ são usados para imprimir strings e caracteres, respectivamente.
- m)** Todas as strings terminam no caractere _____.
- n)** A largura de campo e a precisão em um especificador de conversão de `printf` podem ser controladas com expressões inteiras substituindo um(a) _____ pela largura de campo ou pela precisão, e colocando uma expressão inteira no argumento correspondente da lista de argumentos.
- o)** A flag _____ faz com que a saída seja alinhada à esquerda em um campo.
- p)** A flag _____ faz com que os valores sejam exibidos com um sinal de adição ou com um sinal de subtração.
- q)** A formatação de entrada precisa é obtida a partir da função _____.

- r)** Um(a) _____ é usado(a) para varrer uma string em busca de caracteres específicos e armazenar os caracteres em um array.
- s)** O especificador de conversão _____ pode ser usado para inserir inteiros octais, decimais e hexadecimais opcionalmente com sinal.
- t)** Os especificadores de conversão _____ podem ser usados para a entrada de um valor `double`.
- u)** O _____ é usado para ler dados do stream de entrada e descartá-los sem atribuí-los a uma variável.
- v)** Um(a) _____ pode ser usado(a) em um especificador de conversão de `scanf` para indicar que um número específico de caracteres ou dígitos deve ser lido do stream de entrada.

9.2 Encontre o erro em cada um dos itens a seguir e explique como ele pode ser corrigido.

- a)** A instrução a seguir deve imprimir o caractere 'c'.
`printf( "%s\n", 'c' );`
- b)** A instrução a seguir deve imprimir 9.375%.
`printf( "%.3f%", 9.375 );`
- c)** A instrução a seguir deve imprimir o primeiro caractere da string "Domingo".
`printf( "%c\n", "Domingo" );`
- d)** `printf( ""Uma string entre aspas"" );`
- e)** `printf( %dd, 12, 20 );`
- f)** `printf( "%c", "x" );`
- g)** `printf( "%s\n", 'Ricardo' );`

9.3 Escreva uma instrução para cada um dos itens a seguir:

- a)** Imprimir 1234 alinhado à direita em um campo de 10 dígitos.
- b)** Imprimir 123.456789 em notação exponencial com um sinal (+ ou -) e 3 dígitos de precisão.
- c)** Ler um valor `double` na variável `number`.
- d)** Imprimir 100 em formato octal precedido de 0.
- e)** Ler uma string no array de caracteres `string`.
- f)** Ler caracteres no array `n` até que um caractere diferente de um dígito seja encontrado.
- g)** Usar variáveis inteiras `x` e `y` para especificar a largura de campo e a precisão usadas para exibir o valor `double` 87.4573.
- h)** Ler um valor no formato 3.5%. Armazenar a porcentagem na variável `float percent` e eliminar o % do stream de entrada. Não use o caractere de supressão de atribuição.
- i)** Imprimir 3.333333 como um valor `long double` com um sinal (+ ou -) em um campo de 20 caracteres com uma precisão de 3.

■ Respostas dos exercícios de autorrevisão

- 9.1** a) Fluxos. b) Entrada-padrão. c) Saída-padrão. d) `printf`. e) Especificadores de conversão, flags, larguras de campo, precisões, caracteres literais. f) d, i. g) o, u, x (ou X). h) h, l. i) e (ou E). j) L. k) 6. l) s, c. m) NULL ('\\0'). n) asterisco (*). o) - (subtração). p) + (adição). q) `scanf`. r) Conjunto de varredura. s) i. t) l e, lE, lf, lg ou lG. u) Caractere de supressão de atribuição (*). v) Largura de campo.
- 9.2** a) Erro: O especificador de conversão s espera por um argumento do tipo ponteiro para char.
Correção: Para imprimir o caractere 'c', use o especificador de conversão %c ou mude 'c' para "c".
- b) Erro: Tentativa de imprimir o caractere literal % sem usar o especificador de conversão %%.
Correção: Use %% para imprimir um caractere literal %.
- c) Erro: O especificador de conversão c espera por um argumento do tipo char.
Correção: Para imprimir o primeiro caractere de "Monday", use o especificador de conversão %1s.
- d) Erro: Tentativa de imprimir o caractere literal " sem usar a sequência de escape \".
Correção: Substitua cada aspa no conjunto interno de aspas por \".
- e) Erro: A string de controle de formato não está delimitada por aspas.
Correção: Delimite %d%d com aspas.
- f) Erro: O caractere x está delimitado por aspas.
Correção: As constantes de caractere a serem impressas com %c devem ser delimitadas por aspas simples.
- g) Erro: A string a ser impressa está delimitada por aspas simples.
Correção: Use aspas duplas no lugar de aspas simples para representar uma string.
- 9.3** a) `printf( "%10d\\n", 1234 );`
b) `printf( "%+.3e\\n", 123.456789 );`
c) `scanf( "%lf", &number );`
d) `printf( "%#o\\n", 100 );`
e) `scanf( "%s", string );`
f) `scanf( "[%0123456789]", n );`
g) `printf( "%*.*f\\n", x, y, 87.4573 );`
h) `scanf( "%t%", &percent );`
i) `printf( "%+20.3Lf\\n", 3.333333 );`

■ Exercícios

- 9.4** Escreva uma instrução `printf` ou `scanf` para cada um dos itens a seguir:
- a) Imprimir o inteiro sem sinal 40000 alinhado à esquerda em um campo de 15 dígitos com 8 dígitos.
 - b) Ler um valor hexadecimal na variável `hex`.
 - c) Imprimir 200 com e sem sinal.
 - d) Imprimir 100 em formato hexadecimal precedido por 0x.
 - e) Ler caracteres no array `s` até que a letra `p` seja encontrada.
 - f) Imprimir 1.234 em um campo de 9 dígitos com zeros iniciais.
 - g) Ler um horário no formato hh:mm:ss, armazenando as partes da hora nas variáveis inteiras `hora`, `minuto` e `segundo`. Saltar os sinais de dois pontos (:) no stream de entrada. Usar o caractere de supressão de atribuição.
 - h) Ler uma string no formato "caracteres" a partir da entrada-padrão. Armazenar a string no array de caracteres `s`. Eliminar as aspas do stream de entrada.
 - i) Ler um horário no formato hh:mm:ss, armazenando as partes da hora nas variáveis inteiras `hora`, `minuto` e `segundo`. Saltar os sinais de dois-pontos (:) no stream de entrada. Não use o caractere de supressão de atribuição.
- 9.5** Mostre o que cada uma das instruções a seguir imprime. Se uma instrução estiver incorreta, explique.
- a) `printf( "%-10d\\n", 10000 );`
 - b) `printf( "%c\\n", "Isto é uma string" );`
 - c) `printf( "%*.*lf\\n", 8, 3, 1024.987654 );`
 - d) `printf( "%#o\\n%X\\n#e\\n", 17, 17, 1008.83689 );`
 - e) `printf( "% 1d\\n%+ld\\n", 1000000, 1000000 );`
 - f) `printf( "%10.2E\\n", 444.93738 );`
 - g) `printf( "%10.2g\\n", 444.93738 );`
 - h) `printf( "%d\\n", 10.987 );`
- 9.6** Encontre o(s) erro(s) em cada um dos segmentos de programa a seguir. Explique como cada erro pode ser corrigido.
- a) `printf( "%s\\n", 'Feliz aniversário' );`
 - b) `printf( "%c\\n", 'Olá' );`
 - c) `printf( "%c\\n", "Isso é uma string" );`

- d) A instrução a seguir deveria imprimir “Boa Viagem”
`printf( "%s", "Boa viagem" );`
- e) `char day[] = "Domingo";`
`printf( "%s\n", day[ 3 ] );`
- f) `printf( 'Digite seu nome: ' );`
- g) `printf( %f, 123.456 );`
- h) A instrução a seguir deveria imprimir os caracteres ‘O’ e ‘K’:
`printf( "%s%s\n", 'O', 'K' );`
- i) `char s[ 10 ];`
`scanf( "%c", s[ 7 ] );`

9.7 Diferenças entre %d e %i. Escreva um programa que teste a diferença entre os especificadores de conversão %d e %i quando usados em instruções scanf. Use as instruções

```
scanf( "%i%d", &x, &y );
printf( "%d %d\n", x, y );
```

para aceitar e imprimir os valores. Teste o programa com os seguintes conjuntos de dados de entrada:

10	10
-10	-10
010	010
0x10	0x10

9.8 Exibição de valores e seu número de caracteres. Escreva um programa que carregue o array de 10 elementos number com inteiros aleatórios de 1 a 1000. Para cada valor, imprima o valor e um total acumulado do número de caracteres impressos. Use o especificador de conversão %n para determinar o número de caracteres exibidos para cada valor. Imprima o número total de caracteres exibidos para todos os valores até (e inclusive) o valor atual toda vez que este for impresso.

9.9 Impressão de valores de ponteiro como inteiros. Escreva um programa que imprima valores de ponteiro usando todos os especificadores de conversão de inteiros e o especificador de conversão %p. Quais imprimem valores estranhos? Quais causam erros? Em que formato o especificador de conversão %p exibe o endereço em seu sistema?

9.10 Impressão de números em diversas larguras de campo. Escreva um programa que teste os resultados da impressão do valor inteiro 12345 e o valor de ponto flutuante 1.2345 em diversos tamanhos de campo. O que acontece quando os valores são impressos em campos que contêm menos dígitos do que os valores?

9.11 Arredondamento de números em ponto flutuante. Escreva um programa que imprima o valor 100.453627 arredondado para o dígito mais próximo, décimo, centésimo, milésimo e décimo de milésimo.

9.12 Exibição de uma string em um campo. Escreva um programa que inclua uma string do teclado e determine o

comprimento da string. Imprima a string usando o dobro do tamanho da largura de campo.

9.13 Conversões de temperatura. Escreva um programa que converta temperaturas inteiras em Fahrenheit, de 0 a 212 graus, em temperaturas Celsius de ponto flutuante com 3 dígitos de precisão. Realize o cálculo usando a fórmula

$$\text{celsius} = 5.0 / 9.0 * (\text{fahrenheit} - 32);$$

A saída deverá ser impressa em duas colunas alinhadas à direita com 10 caracteres cada uma, e as temperaturas Celsius devem ser precedidas por um sinal para valores positivos e negativos.

9.14 Sequências de escape. Escreva um programa que teste todas as sequências de escape da Figura 9.16. Para as sequências de escape que movem o cursor, imprima um caractere antes e depois de imprimir a sequência de escape, de modo que fique claro para onde o cursor se moveu.

9.15 Impressão de um ponto de interrogação. Escreva um programa que determine se o símbolo (?) pode ser impresso como parte de uma string de controle de formato printf como um caractere literal, em vez de usar a sequência de escape (\?).

9.16 Leitura de um inteiro com cada um dos especificadores de conversão de scanf. Escreva um programa que receba como entrada o valor 437 usando cada um dos especificadores de conversão de inteiros de scanf. Imprima cada valor inserido usando todos os especificadores de conversão de inteiros.

9.17 Exibição de um número com os especificadores de conversão de ponto flutuante. Escreva um programa que use cada um dos especificadores de conversão e, f e g para inserir o valor 1.2345. Imprima os valores de cada variável para provar que cada especificador de conversão pode ser usado para entrar com esse mesmo valor.

9.18 Leitura de strings entre aspas. Em algumas linguagens de programação, as strings são incluídas delimitadas por aspas simples ou duplas. Escreva um programa que leia as três strings suzy, “suzy” e ‘suzy’. As aspas são ignoradas pela C ou lidos como parte da string?

9.19 Impressão de um ponto de interrogação como uma constante de caractere. Escreva um programa que determine se ? pode ser impresso como uma constante de caractere ‘?’, em vez de uma sequência de escape de constante de caractere ‘\?’, usando o especificador de conversão %c na string de controle de formato de uma instrução printf.

9.20 Uso de %g com diversas precisões. Escreva um programa que use o especificador de conversão g para exibir o valor 9876.12345. Imprima o valor com precisões variando de 1 a 9.